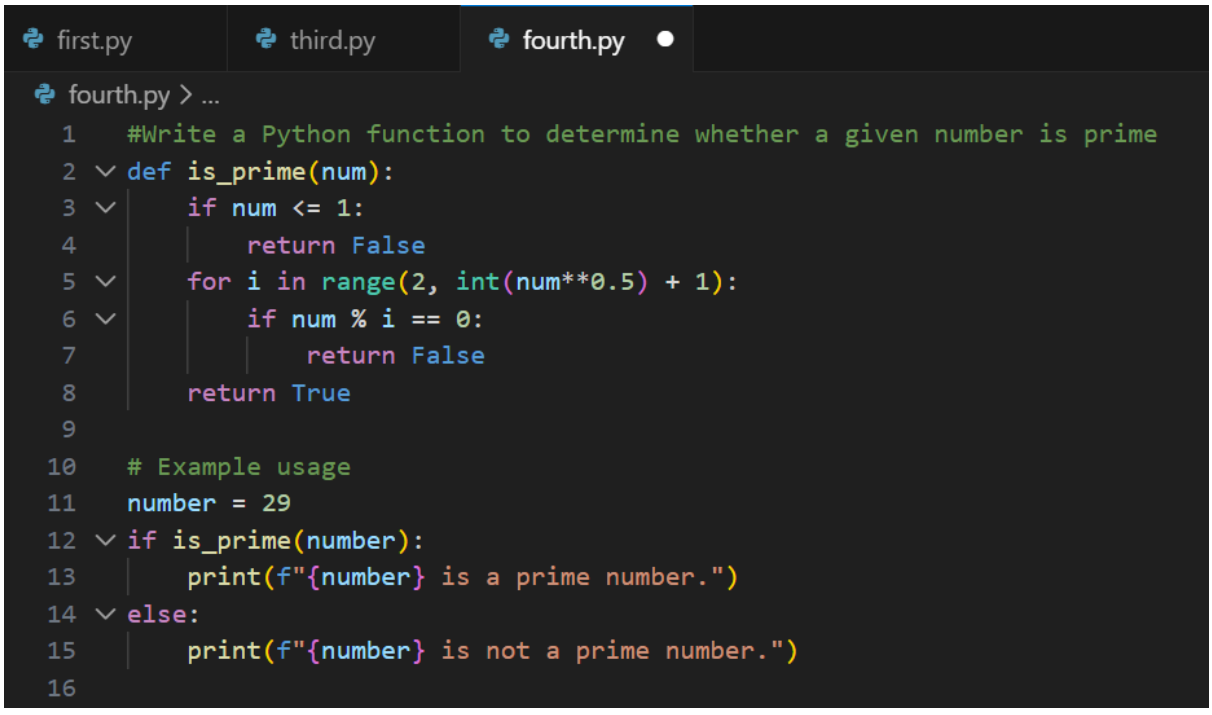
Lab_Assignment_4.2

AI Assisted Coding B37 2303A52077

Task -1

- **Zero-shot: Prompt AI with only the instruction. Write a Python function to determine whether a given number is prime**
- Prompt Used: #Write a Python function to determine whether a given number is prime
- **Code:**
- 

- **Output:** `29 is a prime number.`

Explanation:

This code defines a function called is_prime that checks whether a given number is prime or not. A prime number is a natural number

greater than 1 that cannot be formed by multiplying two smaller natural numbers. The function first checks if the number is less than or equal to 1, in which case it returns False since prime numbers are greater than 1. Then, it iterates from 2 to the square root of the number (inclusive) to check for any divisors. If it finds any number that divides evenly into the input number, it returns False, indicating that the number is not prime. If no divisors are found, it returns True, indicating that the number is prime. The example usage demonstrates how to use the function and prints whether the specified number (29 in this case) is prime or not

Task -2

- One-shot: Provide one example: Input: [1, 2, 3, 4], Output: 10 to help AI generate a function that calculates the sum of elements in a list.
- Prompt used: #write a python function to sum the given list of numbers
- Code:

```
#write a python function to sum the given list of numbers
def sum_of_list(numbers):
    total = 0
    for number in numbers:
        total += number
    return total
# Example usage
num_list = [1, 2, 3, 4, 5]
result = sum_of_list(num_list)
print(f"The sum of the list {num_list} is {result}.")
```

- Output:

```
The sum of the list [1, 2, 3, 4, 5] is 15.
```

Explanation:

This code defines a function called sum_of_list that takes a list of numbers as input and returns the sum of those numbers. The function initializes a variable total to zero and then iterates through each number in the input list, adding each number to the total. After processing all the numbers, it returns the final total. The example usage demonstrates how to use the function by summing a list of numbers [1, 2, 3, 4, 5] and printing the result.

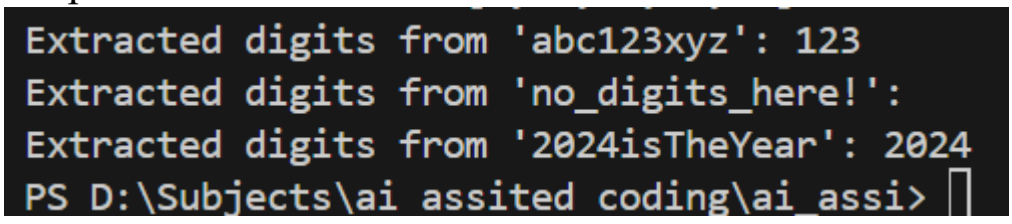
Task -3

- Few-shot: Give 2–3 examples to create a function that extracts digits from an alphanumeric string.
- Prompt used: #write a python function that extracts digits from an alphanumeric string. with 2-3 example usages
- Code:

```python
#write a python function that extracts digits from an alphanumeric string. with 2-3 example usages
def extract_digits(input_string):
    digits = ''.join([char for char in input_string if char.isdigit()])
    return digits
# Example usages
example1 = "abc123xyz"
example2 = "no_digits_here!"
example3 = "2024isTheYear"
print(f"Extracted digits from '{example1}': {extract_digits(example1)}")
print(f"Extracted digits from '{example2}': {extract_digits(example2)}")
print(f"Extracted digits from '{example3}': {extract_digits(example3)}")


```

- Outputs:



Explanation:

This code defines a function called extract_digits that takes an alphanumeric string as input and extracts all the digits from it. The function uses a list comprehension to iterate through each character in

the input string, checking if the character is a digit using the isdigit() method. If it is a digit, it is included in the resulting list. The list of digits is then joined into a single string and returned. The example usages demonstrate how to use the function with different input strings, showing the extracted digits for each case.

Task -4

- Compare zero-shot vs few-shot prompting for generating a function that counts the number of vowels in a string.
- Prompt(zero-shot): #write a python function that counts the number of vowels in a string.
- Code(zero-shot):

```
#write a python function that counts the number of vowels in a string.
def count_vowels(input_string):
    vowels = "aeiouAEIOU"
    count = sum(1 for char in input_string if char in vowels)
    return count
# Example usage
example_string = "Hello, World!"
vowel_count = count_vowels(example_string)
print(f"The number of vowels in '{example_string}' is {vowel_count}.")

```

- Output(zero-shot): The number of vowels in 'Hello, World!' is **3**.
- Prompt(few-shot): #write a python function counts the number of vowels in a string. where be case insensitive, ignores spaces and punctuation.

  #also provide 2-3 example usages,

Code(few-shot):

```python
#write a python function counts the number of vowels in a string. where be case insensitive, ignores spaces and punctuation.
#also provide 2-3 example usages,
import string
def count_vowels_case_insensitive(input_string):
    vowels = "aeiou"
    count = sum(1 for char in input_string.lower() if char in vowels)
    return count
# Example usages
example1 = "Hello, World!"
example2 = "Python is fun."
example3 = "A quick brown fox."
print(f"The number of vowels in '{example1}' is {count_vowels_case_insensitive(example1)}")
print(f"The number of vowels in '{example2}' is {count_vowels_case_insensitive(example2)}")
print(f"The number of vowels in '{example3}' is {count_vowels_case_insensitive(example3)}")

```

Output:

```
The number of vowels in 'Hello, World!' is 3
The number of vowels in 'Python is fun.' is 3
The number of vowels in 'A quick brown fox.' is 5
```

Explanation:

This code defines a function called count_vowels that counts the number of vowels in a given string. The function defines a string containing all the vowels (both lowercase and uppercase) and then uses a generator expression within the sum() function to iterate through each character in the input string. For each character that is found in the vowels string, it adds 1 to the count. Finally, the function returns the total count of vowels. The example usage demonstrates how to use the function by counting the vowels in the string "Hello, World!" and printing the result.

This code defines a function called count_vowels_case_insensitive that counts the number of vowels in a given string while being case insensitive and ignoring spaces and punctuation. The function first converts the input string to lowercase to ensure case insensitivity. It then uses a generator expression within the sum() function to iterate through each character in the lowercase string, checking if the character is in the defined vowels string ("aeiou"). For each vowel

found, it adds 1 to the count. Finally, the function returns the total count of vowels. The example usages demonstrate how to use the function with different input strings, showing the number of vowels for each case.

The first function, count_vowels, counts the number of vowels in a given string by checking each character against a predefined list of vowels (both uppercase and lowercase). It uses a generator expression to sum up the occurrences of vowels in the input string.

The second function, count_vowels_case_insensitive, performs a similar task but is designed to be case insensitive. It converts the input string to lowercase before checking for vowels, ensuring that it counts vowels regardless of their case. Both functions return the total count of vowels found in the input string.

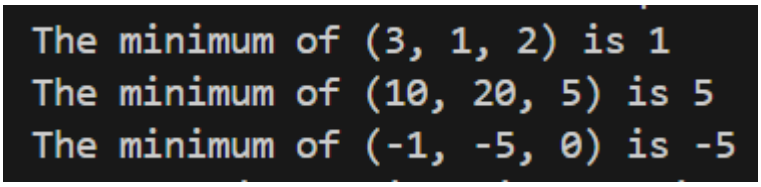
Task Description-5

- Use few-shot prompting with 3 sample inputs to generate a function that determines the minimum of three numbers without using the built-in min() function.
- Prompt: #write a python function that determines the minimum of three numbers without using the built-in min() function. also provide 2-3 example usages
- Code:

```
#write a python function that determines the minimum of three numbers without using the built-in min() function. also provide 2-3 example usages
                (parameter) b: Any
def min_of_three(a, b, c):
    if a <= b and a <= c:
        return a
    elif b <= a and b <= c:
        return b
    else:
        return c
# Example usages
print(f"The minimum of (3, 1, 2) is {min_of_three(3, 1, 2)}")
print(f"The minimum of (10, 20, 5) is {min_of_three(10, 20, 5)}")
print(f"The minimum of (-1, -5, 0) is {min_of_three(-1, -5, 0)}")
```

```
100
101 def min_of_three(a, b, c):
102     if a <= b and a <= c:
103         return a
104     elif b <= a and b <= c:
105         return b
106     else:
107         return c
108 # Example usages
109 print(f"The minimum of (3, 1, 2) is {min_of_three(3, 1, 2)}")
110 print(f"The minimum of (10, 20, 5) is {min_of_three(10, 20, 5)}")
111 print(f"The minimum of (-1, -5, 0) is {min_of_three(-1, -5, 0)}")
112
```

Output:



Explanation:

This code defines a function called min_of_three that determines the minimum of three given numbers without using the built-in min() function. The function compares the three numbers using conditional statements (if, elif, else) to identify which number is the smallest. It returns the smallest number among the three inputs. The example usages demonstrate how to use the function with different sets of numbers, printing the minimum value for each case.